



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**PROYECTO "APLICACIÓN MÓVIL COLABORATIVA PARA
MEJORAR LA EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS
PERDIDAS CON PARTICIPACIÓN DE VOLUNTARIOS EN LA
CIUDAD DE TACNA, 2026"**

Curso: Construcción de software I

Docente: RICARDO EDUARDO VALCARCEL ALVARADO

Integrantes:

Chite Quispe Brian Danilo (2021070015)

Anampa Pancca David Jordan (2022074268)

**Tacna – Perú
2026**

ÍNDICE

1.	Introducción.....	3
2.	Estándares de Arquitectura.....	3
3.	Estándares de Codificación C#.....	4
4.	Estándares de Base de Datos.....	6
5.	Estándares de Nomenclatura.....	6
6.	Estándares de Interfaz de Usuario.....	7
7.	Estándares de Documentación.....	8
8.	Estándares de Seguridad.....	8
9.	Referencias.....	11

1. INTRODUCCIÓN

El presente documento establece los estándares de programación utilizados en el desarrollo del sistema SOSMASCOTA, una aplicación móvil colaborativa orientada a la asistencia en la búsqueda de mascotas perdidas en la ciudad de Tacna.

Estos estándares tienen como finalidad asegurar la correcta organización del código, la mantenibilidad del sistema y la coherencia en el desarrollo del proyecto, considerando el uso de tecnologías como Flutter y Firebase bajo una arquitectura basada en el patrón MVC adaptado a aplicaciones móviles.

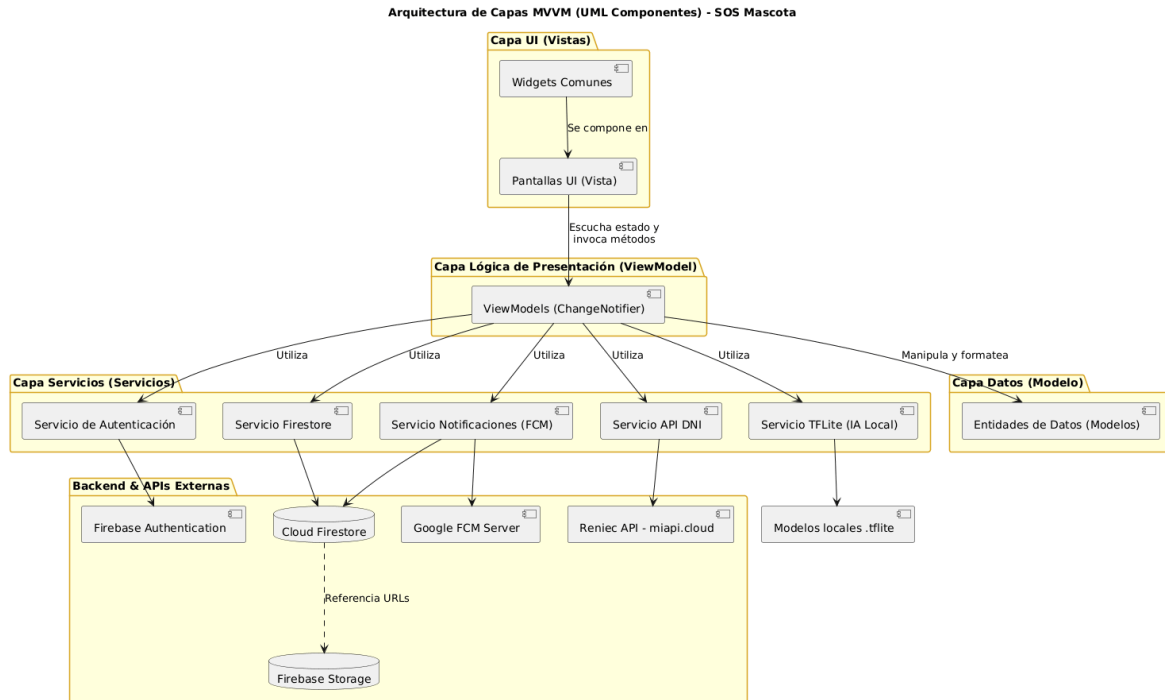
El cumplimiento de estas normas permite que el equipo de desarrollo trabaje de manera ordenada, facilitando la integración de módulos, la reutilización de código y la escalabilidad del sistema. Asimismo, se establecen convenciones para nomenclatura, estructura de carpetas, manejo de base de datos, seguridad, control de versiones y buenas prácticas de programación.

De esta manera, los estándares definidos garantizan un desarrollo limpio, organizado y alineado a prácticas profesionales de ingeniería de software, asegurando la calidad del producto final.

2. ESTÁNDARES DE ARQUITECTURA

- El sistema SOS Mascota no utiliza un patrón MVC tradicional. En su lugar, implementa el patrón MVVM (Model-View-ViewModel) adaptado a Flutter y reactividad mediante inyección de dependencias con `ChangeNotifierProvider` de `Provider`.

2.1 Diagrama UML de Capas del Sistema (MVVM)



2.2 Estructura del Proyecto (lib/)

El árbol de directorios del proyecto se divide estrictamente de acuerdo con el patrón MVVM y las responsabilidades del sistema:

lib/

└─ modelo/	# Capa Modelo: Clases de datos y serializadores (fromMap/toMap)
└─ vista/	# Capa Vista: Interfaces de usuario estructuradas por módulos
└─ auth/	# Vistas del flujo de ingreso y registro
└─ usuario/	# Vistas del perfil e interacciones comunitarias
└─ reportes/	# Formularios y listas de mascotas y avistamientos
└─ admin/	# Vistas del dashboard de gestión administrativa
└─ mapa/	# Vista del mapa interactivo (OSM)

└─ chat/	# Vistas de chats y comunicación de salas
└─ vistamodelo/ (ChangeNotifier)	# Capa ViewModel: Controladores de estado
└─ auth/	
└─ reportes/	
└─ admin/	
└─ usuario/	
└─ notificacion/	
└─ comentarios/	
└─ servicios/ IA	# Capa Servicios: Comunicaciones de red, APIs e inferencia
└─ widgets/	# Widgets compartidos y componentes UI reutilizables
└─ utils/	# Clases de soporte, wrappers de autenticación y helpers
└─ app.dart	# Configuración global del MaterialApp y tabla de rutas
└─ main.dart locales	# Punto de entrada de la aplicación e inicializaciones

2.3 Capas del Sistema

- **Modelo (lib/modelo/):** Clases Dart nativas que mapean la información de la base de datos Firestore (e.g., Usuario, ReporteMascota, Avistamiento). Contienen métodos factoría para parsing de datos (fromMap) y exportación (toMap).

- **Vista (lib/vista/):** Widgets estáticos (StatelessWidget) o dinámicos (StatefulWidget) responsables únicamente de renderizar la interfaz. Consumen datos de los ViewModels mediante selectores de lectura o widgets constructores (Consumer).
- **ViewModel (lib/vistamodelo/):** Clases Dart que extienden de ChangeNotifier. Manejan la lógica de la UI, los controladores de campos de texto (TextEditingController), banderas de carga (cargando) y llaman a los servicios de red. Notifican cambios a la UI usando notifyListeners().
- **Servicio (lib/servicios/):** Capa de abstracción que interactúa con agentes externos (Firebase Authentication, Cloud Firestore, Firebase Storage, TFLite local, HTTP requests).

3. ESTÁNDARES DE CODIFICACIÓN (FLUTTER / DART)

3.1 Nomenclatura del Código

Clases y Enums: Utilizan la convención PascalCase.

Código:

```
class ReporteMascotaVM extends ChangeNotifier {}
```

```
class ServicioTFLite {}
```

Variables, Métodos y Parámetros: Utilizan la convención camelCase.

Código:

```
String montoRecompensa = "";
```

```
Future<String> subirFoto(File archivo) async {}
```

Constantes: Utilizan la convención camelCase (según las guías oficiales de Dart).

Código:

```
const umbralDeseado = 0.95;
```

```
const jsonKeyPath = 'assets/keys/service_account.json';
```

Nombres de Archivos (.dart): Utilizan estrictamente snake_case en minúsculas.

```
pantalla_reporte_mascota.dart
```

```
reporte_mascota.dart
```

```
reporte_vm.dart
```

```
auth_servicio.dart
```

3.2 Convenciones Específicas de la Vista (Flutter)

- Todas las pantallas principales deben llevar el prefijo Pantalla en su clase, y pantalla_ en el archivo.

Código:

```
class PantallaLogin extends StatelessWidget {}
```

```
class PantallaAdminUsuarios extends StatefulWidget {}
```

- Los ViewModels de soporte deben llevar el sufijo VM en la clase, y _vm en el nombre de archivo.

Código:

```
class RegistroVM extends ChangeNotifier {}
```

- Los servicios deben llevar el sufijo Servicio en la clase, y _servicio en el nombre de archivo.

Código:

```
class NotificacionServicio {}
```

4. ESTÁNDARES DE BASE DE DATOS (FIREBASE FIRESTORE)

En Cloud Firestore, al ser una base de datos NoSQL documental, no se emplean tablas relacionales tradicionales. Se aplican las siguientes directrices de nomenclatura y modelado:

- **Colecciones y Subcolecciones:** Se nombran en snake_case en minúsculas y en plural, con coherencia lógica.
 - **Correcto:** usuarios, reportes_mascotas, avistamientos, comentarios.
 - **Incorrecto:** Users, Pets, LostReports, FoundReports.
- **Campos y Propiedades:** Se nombran obligatoriamente en camelCase.
 - **Ejemplo:** fotoPerfil, montoRecompensa, fechaRegistro, estadoRol.
- **IDs de Documentos:**
 - Para la colección usuarios, el ID de documento debe ser idéntico al UID generado por Firebase Authentication.
 - Para el resto de colecciones (reportes_mascotas, avistamientos, chats, comentarios), se emplean identificadores autogenerados por el SDK de Firestore (doc().id).

5. ESTÁNDARES DE INTERFAZ DE USUARIO (UI/UX)

- **Directrices Visuales:** Uso del ecosistema de componentes de Material Design 3 con un seed color consistente (ColorScheme.fromSeed(seedColor: Color(0xFF4D9EF6))).
- **Layout Adaptativo:** Uso del paquete responsive_framework para soportar de manera fluida pantallas de teléfonos con distintas relaciones de aspecto e interfaces web/desktop.
- **Gestión de Estado:** Queda terminantemente prohibido llamar a setState() de forma indiscriminada para lógica de negocio. setState() se reserva exclusivamente para animaciones locales y estados transitorios de UI (e.g.

índices de pestañas). La lógica y datos del negocio se inyectan y leen a través de Provider.

- **Interactividad y Carga:**

- Las peticiones asíncronas deben mostrar obligatoriamente un componente de feedback visual (CircularProgressIndicator o LinearProgressIndicator).
- Los botones deben desactivarse lógicamente mientras se procesan operaciones de red para evitar envíos dobles.

6. ESTÁNDARES DE SEGURIDAD Y VALIDACIONES

6.1 Reglas de Seguridad en Cloud Firestore (firestore.rules)

El sistema debe operar bajo el principio de menor privilegio. No se permiten lecturas o escrituras abiertas globales (allow read, write: if true;).

- Las reglas de seguridad validan la autenticación (`request.auth != null`) y la propiedad del recurso (`resource.data.usuarioId == request.auth.uid`).
- Los administradores del sistema (`rol == 'admin'`) tienen permisos explícitos de borrado y actualización en colecciones críticas, definidos mediante consultas de resolución al documento del usuario:

Código: javascript

```
function esAdmin() {

    return request.auth != null &&

    get(/databases/$(database)/documents/usuarios/$(request.auth.uid)).data.rol
    == 'admin';

}
```

6.2 Validaciones del Lado del Cliente

- **Campos de Texto:** Uso obligatorio de Form y TextFormField con sus respectivos parámetros validator.
- **Validación de Imágenes (IA Local):** Antes de procesar la subida de un recurso a Firebase Storage, la imagen debe pasar por la inferencia local de ServicioTFLite.detectarAnimal() y superar un umbral de confianza ≥ 0.95 de ser perro o gato. De lo contrario, se debe rechazar y alertar al usuario antes de consumir ancho de banda de red.

7. CONTROL DE VERSIONES (GIT / GITHUB)

- **Ramas del Repositorio:**
 - **main / master:** Código estable en producción.
 - **develop:** Código de integración de características validadas.
 - **feature/nombre-funcionalidad:** Ramas de desarrollo temporal para nuevos requerimientos (e.g., feature/login, feature/tflite-matching).
 - **bugfix/nombre-error:** Ramas destinadas a corregir incidencias de código.
- **Commits:** Deben ser atómicos y usar prefijos claros en español:
 - **feat:** agrega motor de coincidencia por embeddings de IA
 - **fix:** corrige error de desborde de pantalla en el formulario de avistamientos
 - **docs:** actualiza el manual de estándares de codificación

8. ESTÁNDARES DE DOCUMENTACIÓN Y PRUEBAS

- **DartDoc:** Los métodos, clases y propiedades públicas críticas deben documentarse obligatoriamente utilizando el triple slash (///) para generar autocompletado en el IDE.

Código:

/// Compara dos imágenes de mascotas usando la similitud coseno

/// de los vectores de embeddings generados por TFLite.

/// Retorna un valor de coincidencia de [0.0] a [1.0].

```
static Future<double> compararImagenes(File img1, File img2) async { ...
}
```

- **Testing de Regresión:** Las validaciones críticas e integraciones se prueban unitariamente utilizando flutter_test y simuladores de bases de datos mediante los paquetes mockito, mocktail y fake_cloud_firestore.

9. REFERENCIAS

- Flutter Documentation: <https://flutter.dev>
- Firebase Documentation: <https://firebase.google.com>
- Google Maps API: <https://developers.google.com/maps>
- Dart Language: <https://dart.dev>

Resumen de Estándares Clave

- **Nomenclatura:** Se utiliza PascalCase para clases en Dart, camelCase para variables y funciones, snake_case para nombres de archivos Dart, y convenciones claras en Firebase, donde los campos utilizan camelCase y las colecciones se definen en PascalCase.
- **Arquitectura:** El sistema implementa el patrón MVC adaptado a Flutter, con una separación clara entre Modelos, Vistas (Screens) y Controladores/Services, permitiendo una mejor organización del código y acceso eficiente a Firebase.
- **Seguridad:** Se implementa autenticación mediante Firebase Authentication, reglas de seguridad en Firestore, validación de datos tanto en el frontend como en el backend, y control de acceso basado en roles de usuario y administrador.
- **Base de Datos:** Se utiliza Firebase Firestore como base de datos NoSQL, estructurada en colecciones y documentos que representan entidades como usuarios, reportes, chats y notificaciones.

- Control de Versiones: Se emplea Git y GitHub para el control de versiones, utilizando commits descriptivos y ramas organizadas por funcionalidades (feature/login, feature/reports, entre otras).
- Documentación: Se aplican comentarios en formato DartDoc para funciones y clases principales, además de documentación inline en lógica compleja para mejorar la comprensión del código.
- Testing: Se realizan pruebas básicas mediante flutter_test para validación de lógica, complementadas con pruebas manuales en funcionalidades críticas del sistema integrado con Firebase.
- Convenciones: Convenciones: Código limpio (Clean Code), principios DRY (No repetir código), y buenas prácticas de SOLID aplicadas a Flutter y separación de servicios.

Proyecto: SOSMASCOTA - Aplicación móvil colaborativa para la asistencia de mascotas perdidas con participación de voluntarios en la ciudad de Tacna, 2026.

Tecnologías: Flutter, Firebase (Authentication, Firestore, Cloud Messaging, Storage), Google Maps API, Android.

Versión: 3.0

Fecha: 05 de Julio 2026